

共享计数方法与通用驻留端点

INT8 逻辑参考、128×128 局部映射与四类矩阵模板

2026 年 9 月 24 日

状态：Step 1 已通过；Step 2 完成，待审阅。 Step 1 审阅版本为 e95de9e，本轮从干净的 d60bc32 工作区继续。本文固定共同计数规则，完成 Table II(a)；Table II(b) 仅给符号模板和合成检查，未代入六模型形成正式结果。

1 分析前提、精度与算子交接

固定对象 一次使用是一个输入向量求值。Table II(a) 的动态端点每次使用前装载完整矩阵；FFN 行的一次驻留则装载一次、服务 $B \in \{8, 64, 512\}$ 个向量。Attention 的 $L \in \{1024, 16384, 131072\}$ 是本次追加后的可见长度。六模型、revision 与选定的完整/全局 GQA 子层沿用 Step 1，均不乘主干层数。

计量对象 Q_S 是窗口内进入声明 streaming 输入边界并被服务的逻辑字节； Q_R 是窗口内建立、更新或重载可计算 resident 状态的逻辑写入字节 [1]。当 $Q_R > 0$ 时 $RI = Q_S/Q_R$ ；有求值且 $Q_R = 0$ 时 $RI = \infty$ 。空窗口的 0/0 仅作递推检查，不给 RI 值。

本轮新选的精度参考 所有被计矩阵输入与 resident 数值均以每元素 1 Byte 的有符号 INT8 表示；Attention 系数使用其非负子域。符号推导仍分角色保留宽度：

符号	数值角色	参考值 (Byte/element)
b_x, b_z	投影/FFN 输入；门控乘积形成的 down 输入	1
b_q, b_p	QK 求值的 query；softmax 后的 Attention 系数	1
b_W	通用矩阵权重	1
b_g, b_u, b_d	FFN gate、up、down 三矩阵权重	1
b_K, b_V	resident Key 与 Value	1

这是与 Task I 的 INT8 逻辑合同对齐的研究参考，不改写已保存的原生 BF16/FP8 事实，也不构成六模型 INT8 精度验证。定标参数在数字接口预置，在所声明层/头/角色的窗口内保持固定；本参考计数不含定标元数据。

阶段交接 局部输出先在数字域做足够宽的部分和累加，再完成适用的归一化、RoPE、softmax、SiLU、逐元素乘法或 Value 缩放，最后按下一角色重定标/量化。本次矩阵输出不直接加入 Q_S ；下次进入另一矩阵阶段时才计该阶段的输入。位串行、ADC 转换、编码、verify、restore 等属于硬件服务，不重复增加逻辑 payload。数字衔接与元数据虽不计入这里的矩阵数值流量，仍需真实实现的时间和资源。

2 两种边界与共同驻留映射

主口径与对照口径 上标 loc 表示所有 128×128 tile 的输入接收端累计, 用于与 Task I 的局部服务合同衔接; 上标 op 表示整矩阵阶段入口, 用于解释和对照。同一阶段、同一源向量向并行分支分发时, op 只计一次上游输入; loc 对每个实际 tile 接收分别计数。串行阶段的新输入再次计入, 因此 FFN 的 op 仍含 down 输入, 并非整个 FFN 外部 I/O 的唯一张量大小。

分块与接收 对 $W \in \mathbb{Z}^{N \times K}$, N 是输出行、 K 是输入列。定义

$$c_N(x) = \left\lceil \frac{x}{T_N} \right\rceil, \quad c_K(x) = \left\lceil \frac{x}{T_K} \right\rceil, \quad T_N = T_K = 128. \quad (1)$$

输出块 a 、输入块 b 的有效尺寸为 $n_a = \min(T_N, N - aT_N)$ 、 $k_b = \min(T_K, K - bT_K)$ 。每个 (a, b) tile 常驻 $W_{a,b}$, 一次向量求值接收 k_b 个输入; 同一输入分片可从上游缓冲广播, 但不同 a 的接收端分别计费。

装载、求值与部分和 动态驻留先把每个有效 tile 装载一次, 随后按输出块、输入块的顺序枚举求值调用, 并在数字累加器中归并同一输出块的部分和。全部所需 tile 在声明驻留期内保留; 静态权重装载在窗口之前。该枚举是逻辑调用清单, 不规定所有 tile 并行或某个最优时序。参考有足够的逻辑服务单元容纳所声明状态, 不引入容量不足的重载。部分和不重新作为 CIM 输入, 输入分块也不等于新增 bit-plane payload。

可直接执行的通用式 记通用输入与 resident 元素宽度为 b_S, b_R (Byte/element)。若窗口内有 U 次向量使用、每个 resident tile 有 A 次完整逻辑装载, 则

$$Q_S^{\text{op}} = UKb_S, \quad Q_S^{\text{loc}} = U \sum_a \sum_b k_b b_S = U c_N(N) K b_S, \quad (2)$$

$$Q_R^{\text{op}} = Q_R^{\text{loc}} = A \sum_a \sum_b n_a k_b b_R = ANK b_R, \quad C_{\text{eval}} = U c_N(N) c_K(K). \quad (3)$$

保留每种 (n_a, k_b) 的有效形状和调用次数; 逻辑 tile 装载项数是 $A c_N(N) c_K(K)$, 不等于器件写事务数。尾块只计有效元素, 不据少输入字节推断完整服务时间同比缩短。

合成例与结论 仅为检查, 令 $T_N = 3, T_K = 2, N = 5, K = 7, U = A = 1$ 、每元素 1 Byte。有效输出块为 $(3, 2)$ 、输入块为 $(2, 2, 2, 1)$, 共 8 个 tile; 得到 $(Q_S^{\text{op}}, Q_R^{\text{op}}) = (7, 35)$ Byte、 $(Q_S^{\text{loc}}, Q_R^{\text{loc}}) = (14, 35)$ Byte。改变为 $U = 4, A = 1$ 时, 输入随使用次数变成 28/56 Byte, resident 写入仍为 35 Byte。

与硬件能力的对应 累计需求描述一个明确的 tile 集合, 不能直接除以单 macro 能力当成完成时间。之后若进行硬件比较, 必须对同一集合和精度声明资源归约、共享/互斥、尾块利用率及写入模式; 本轮只给需求, 不重估 Task I。

3 Table II(a): 两种驻留端点

前提 窗口只覆盖一个输入向量, 故 $U = 1$ 。Weight-static 取 $A = 0$, 权重在窗口外预驻留; Per-use reloaded 取 $A = 1$, 窗口内完整装载一次再求值。后者不是完整训练 step, 也不把多向量 GEMM 放进这次使用。由式(2)–(3) 得到

驻留端点	整算子阶段入口 ($Q_S^{\text{op}}, Q_R^{\text{op}}, \text{RI}^{\text{op}}$)	局部接收端 ($Q_S^{\text{loc}}, Q_R^{\text{loc}}, \text{RI}^{\text{loc}}$)
Weight-static	$(Kb_S, 0, \infty)$	$(c_N(N) Kb_S, 0, \infty)$
Per-use reloaded	$\left(Kb_S, NKb_R, \frac{b_S}{Nb_R}\right)$	$\left(c_N(N) Kb_S, NKb_R, \frac{c_N(N) b_S}{Nb_R}\right)$

这两套三元组有不同输入边界; 共享的 Q_R 只表示本参考没有复制权重或额外重载。

精确数值 下表取 $b_S = b_R = 1$ 。写入列仅对应动态端点; 静态端点的输入列相同, $Q_R = 0$ 、 $\text{RI} = \infty$ 。

(N, K)	tile/调用数	Q_S^{op} (Byte)	Q_S^{loc} (Byte)	动态 Q_R (Byte)	动态 RI^{op}
(128, 128)	1	128	128	16 384	1/128
(1024, 1024)	64	1 024	8 192	1 048 576	1/1024
(4096, 4096)	1024	4 096	131 072	16 777 216	1/4096
(1024, 4096)	256	4 096	32 768	4 194 304	1/1024
(4096, 1024)	256	1 024	32 768	4 194 304	1/4096

$$\text{RI}_{\text{reload}}^{\text{loc}} = \frac{1}{128} = 0.0078125 \quad (\text{五个规定形状均成立}). \quad (4)$$

为何相同 五种形状的 N 都是 128 的整数倍, 因此 $c_N(N) = N/128$ 。固定一次使用, 局部输入为 $NK/128$ Byte, 完整装载为 NK Byte, 比例即 1/128。例如 1024×4096 和 4096×1024 的局部输入都为 32 KiB、写入都为 4 MiB; 整算子输入却分别为 4 KiB 与 1 KiB。规模不同不必导致比例不同; 这里没有为制造差异更改边界。

尾块说明 对非整除 N , 动态 $\text{RI}^{\text{loc}} = \lceil N/128 \rceil / N$ (等字节宽度), 不强行套用 1/128。输入方向的尾块不增加有效逻辑总长 K , 但调用数仍包含其独立服务。

交付入口 英文主表采用局部接收端口径, 逐格列 Q_S, Q_R, RI 。table_IIa/data/results.json 保存两套完整精确数据; table_IIa/tex/table_fragment.tex 是生成片段, table_IIa/tex/table_IIa.tex 是独立英文入口。形状只定义逻辑任务, 不重新定义五种物理 macro。

4 QKV Projection: 原生矩阵与统一端口规则

前提与布局选择 输入为一个 D 维 token 向量，投影权重预驻留。本参考将语义矩阵 Q 、可选的 Attention output gate G 、 K 、 V 分别切成 tile bank，**不跨语义矩阵合并尾块**。源码中的融合张量可按语义切片/重排；这一存储动作不复制权重，也不自动合并 CIM 接收端。因此 fused QKV 与独立 Q/K/V 都遵循同一规则。

令 H_q, H_{kv}, d_{QK}, d_V 为原生头数和维度， D_g 为额外 output gate 宽度。无门控时 $D_g = 0$ ；两款选定 Qwen 的 $D_g = H_q d_V$ ，其原生 q-projection 不能缩成纯 Q[5, 6]。

矩阵	有效形状 $N \times K$	单独入口 Q_S^{op}	接收端 Q_S^{loc}
W_Q	$(H_q d_{QK}) \times D$	Db_x	$c_N(H_q d_{QK}) Db_x$
W_G (若有)	$D_g \times D$	Db_x	$c_N(D_g) Db_x$
W_K	$(H_{kv} d_{QK}) \times D$	Db_x	$c_N(H_{kv} d_{QK}) Db_x$
W_V	$(H_{kv} d_V) \times D$	Db_x	$c_N(H_{kv} d_V) Db_x$

每项 $Q_R = 0, \text{RI} = \infty$ ， W_G 不存在时整项省略。

汇总与共享输入 设实际非空矩阵集合为 $\mathcal{J}$ ， $m = |\mathcal{J}|$ ，各自输出宽度为 n_j 。同一个 token 输入在整算子阶段入口只进入一次，而每个 tile bank 接收端独立：

$$Q_{S, \text{QKV}}^{\text{op}} = \sum_{j \in \mathcal{J}} Db_x - (m - 1) Db_x = Db_x, \quad (5)$$

$$Q_{S, \text{QKV}}^{\text{loc}} = Db_x \sum_{j \in \mathcal{J}} c_N(n_j), \quad Q_{R, \text{QKV}} = 0, \quad (6)$$

$$C_{\text{QKV}} = c_K(D) \sum_{j \in \mathcal{J}} c_N(n_j), \quad \text{RI}_{\text{QKV}}^{\text{op}} = \text{RI}_{\text{QKV}}^{\text{loc}} = \infty. \quad (7)$$

上表的 op 分项是独立矩阵入口，有同源重叠，汇总时已显式扣除；loc 分项的接收端不同，可以直接相加。只按数值相等而去重不同 token/heads 的输入不属于本规则。

合成检查 设 $D = 5, H_q = 4, H_{kv} = 2, d_{QK} = 3, d_V = 2, D_g = 8$ ，测试 tile 为 3×2 。四项输出宽度为 12, 8, 6, 4，对应 4, 3, 2, 2 个输出块。故 $Q_S^{\text{op}} = 5$ Byte、 $Q_S^{\text{loc}} = 55$ Byte、调用数 $3(4 + 3 + 2 + 2) = 33$ ， $Q_R = 0$ 。没有 gate 时局部输入为 40 Byte；额外门控的投影需求已明确保留。

算子交接与结论 Q/K 的归一化和 RoPE、gate 的 sigmoid 与逐元素乘法在数字域处理；gate 本身不成为 KV 缓存，也不变成新的 query head。本行仅含原生投影，不含 Attention 输出投影 W_O ，不把 KV 写入混入预驻留投影权重的 Q_R 。KV 建立/追加另由下文 Attention 窗口计量。各行是独立场景，不能不检查窗口就拼成端到端总量。

5 FFN / MoE: 一次装载后的实际向量复用

前提 只分析一个 Dense FFN 或一个路由专家的完整 SwiGLU:

$$z = \text{SiLU}(W_g x) \odot (W_u x), \quad y = W_d z, \quad (8)$$

其中 $W_g, W_u \in \mathbb{Z}^{F \times D}$ 、 $W_d \in \mathbb{Z}^{D \times F}$ [4, 6]。窗口内三矩阵各装载一次，服务 B 个实际输入向量。三个 tile bank 同期保留，无专家替换的额外重载；窗口结束后替换不在本窗口重复收费。

分项需求 z 是数字域 SiLU 和逐元素乘积完成后，重定标成宽度 b_z 的新矩阵输入。

矩阵	单独入口 Q_S^{op}	接收端 Q_S^{loc}	Q_R (两边界一致)
W_g	BDb_x	$Bc_N(F)Db_x$	FDb_g
W_u	BDb_x	$Bc_N(F)Db_x$	FDb_u
W_d	BFb_z	$Bc_N(D)Fb_z$	DFb_d

每项 RI 按本行对应两列求商，例如

$$\text{RI}_g^{\text{op}} = \frac{Bb_x}{Fb_g}, \quad \text{RI}_g^{\text{loc}} = \frac{Bc_N(F)b_x}{Fb_g}, \quad \text{RI}_d^{\text{loc}} = \frac{Bc_N(D)b_z}{Db_d}. \quad (9)$$

up 将 b_g 换为 b_u ，down 的整算子式为 $Bb_z/(Db_d)$ 。

汇总公式 gate/up 使用同一份 x ，其上游输入可共享；down 使用另一个 z 。两组阶段入口之间不把 $W_g x, W_u x$ 输出再各加一份，只在 down 真正接收 z 时计入：

$$Q_{S,\text{FFN}}^{\text{op}} = B(Db_x + Fb_z), \quad (10)$$

$$Q_{S,\text{FFN}}^{\text{loc}} = B(2c_N(F)Db_x + c_N(D)Fb_z), \quad (11)$$

$$Q_{R,\text{FFN}} = DF(b_g + b_u + b_d), \quad (12)$$

$$C_{\text{FFN}} = B(2c_N(F)c_K(D) + c_N(D)c_K(F)). \quad (13)$$

式(10)从独立分项之和扣除一次共享的 BDb_x ；式(11)保留两个 bank 的接收。两种汇总 RI 都由对应的累计 Q_S/Q_R 得到，不能平均分项 RI。

合成例 令 $D = 5, F = 7, B = 4$ ，测试 tile 为 3×2 ，所有字节宽度为 1。局部分项输入为 gate/up/down = (60, 60, 56) Byte，三项写入各 35 Byte。于是局部汇总为 (176, 105, 176/105)，整算子汇总为 (48, 105, 16/35)。增加 B 只增加使用次数和输入接收；本驻留期的完整装载仍只有一次。

对象范围 MoE 的 top-k、全部路由专家、共享专家和其它层均不作本行隐式倍数。路由矩阵不属于所选专家的 gate/up/down；也不设路由均匀、命中率或额外切换假设。 B 是同一份专家权重真正服务的向量总数，不是批次数或全局 batch size。

6 Attention: 两种 resident 方向与单前缀规则

头参数与 GQA 保留独立的 H_q, H_{kv}, d_{QK}, d_V , 令 $g = H_q/H_{kv}$ 为整数。每个 KV head 只持有一份 K/V, 组内 g 个 query heads 依次对同一 resident 状态求值 [2]。每个 query head 有自己的 query 和 Attention 系数, streaming 按各次求值计数; 源码中的 `repeat_kv` 不形成额外持久副本。

固定状态坐标与方向 对 KV head h 、可见前缀 i , 把每个 token 的 Key 和 Value 分别排列为

$$\mathcal{K}_i^{(h)} = \begin{bmatrix} k_1^{(h)\top} \\ \vdots \\ k_i^{(h)\top} \end{bmatrix} \in \mathbb{R}^{i \times d_{QK}}, \quad \mathcal{V}_i^{(h)} = \begin{bmatrix} v_1^{(h)} & \dots & v_i^{(h)} \end{bmatrix} \in \mathbb{R}^{d_V \times i}. \quad (14)$$

列向量写法下, $s_i^{(q)} = \mathcal{K}_i^{(h)} q_i^{(q)}$, $o_i^{(q)} = \mathcal{V}_i^{(h)} p_i^{(q)}$, 与常见行向量 $QK^\top$ 和 PV 等价 [3]。**K 的序列方向是输出行, V 的序列方向是输入列。**因此序列增大导致 QK 的 query 向更多输出块重放, 而 AV 的系数向量本身变长。

因果前缀的求值组织 Prefill 从空本段状态开始: 对 $i = 1, \dots, L$, 先将 token i 的 K/V 各写入一次, 再对全部 H_q 个 query heads 求值可见前缀 $1:i$; 计算所有有效 scores 后做因果 softmax, 再求值 AV。这是完整的逐前缀逻辑参考, 不声称是推理引擎的最快 Prefill 调度。旧 token 不移动、不压缩、不重新装载; 未来未写入位置不参加本次求值。

一个前缀的输入与调用 同一 token 下所有 query heads 合计:

$$Q_{S,QK}^{\text{op}}(i) = H_q d_{QK} b_q, \quad Q_{S,QK}^{\text{loc}}(i) = H_q d_{QK} b_q c_N(i), \quad (15)$$

$$Q_{S,AV}^{\text{op}}(i) = H_q i b_p, \quad Q_{S,AV}^{\text{loc}}(i) = H_q i b_p c_N(d_V), \quad (16)$$

$$C_{QK}(i) = H_q c_N(i) c_K(d_{QK}), \quad C_{AV}(i) = H_q c_N(d_V) c_K(i). \quad (17)$$

Q_S 只包含 query 和下一次求值入口的系数 p 。scores 是第一次输出, 不另加到 Q_S 。softmax 需要的数字保存与处理、RoPE/归一化及 Value 缩放均在矩阵接口交接。

追加的形状与写入 每个 KV head 追加 K 的 $1 \times d_{QK}$ 行、V 的 $d_V \times 1$ 列:

$$\Delta Q_{R,K} = H_{kv} d_{QK} b_K, \quad \Delta Q_{R,V} = H_{kv} d_V b_V. \quad (18)$$

K 行分散到 $c_K(d_{QK})$ 个 $1 \times k_b$ 写入片; V 列分散到 $c_N(d_V)$ 个 $n_a \times 1$ 写入片。在预分配的绝对 token 坐标上增长有效形状, 跨 tile 边界时只写新元素, 不把所有小更新补成整 tile 重写。这些逻辑形状尚不保证任何介质的物理 append 效率。

7 Prefill 与 Decode 的精确闭式模板

因果求和 从相同前缀规则精确累加，定义

$$P(L) = \sum_{i=1}^L i = \frac{L(L+1)}{2}, \quad (19)$$

$$S_T(L) = \sum_{i=1}^L \left\lceil \frac{i}{T} \right\rceil = \frac{Tm(m+1)}{2} + (m+1)r, \quad L = mT + r, \quad 0 \leq r < T. \quad (20)$$

每个完整的 T 项区间贡献 $T, 2T, \dots, mT$ ，余下 r 项各贡献 $m+1$ 。这里不以 $L^2/2$ 替代 $P(L)$ ，也不以连续近似代替 ceiling 求和。

Prefill: 从空 KV 到长度 L 两次矩阵任务的分项与汇总为

$$Q_{S,QK}^{\text{op}} = H_q L d_{QK} b_q, \quad Q_{S,QK}^{\text{loc}} = H_q d_{QK} b_q S_{T_N}(L), \quad (21)$$

$$Q_{S,AV}^{\text{op}} = H_q b_p P(L), \quad Q_{S,AV}^{\text{loc}} = H_q c_N(d_V) b_p P(L), \quad (22)$$

$$Q_{R,K} = L H_{kv} d_{QK} b_K, \quad Q_{R,V} = L H_{kv} d_V b_V, \quad (23)$$

$$Q_{S,\text{pre}}^{\text{op}} = H_q [L d_{QK} b_q + b_p P(L)], \quad (24)$$

$$Q_{S,\text{pre}}^{\text{loc}} = H_q [d_{QK} b_q S_{T_N}(L) + c_N(d_V) b_p P(L)], \quad (25)$$

$$Q_{R,\text{pre}} = L H_{kv} (d_{QK} b_K + d_V b_V), \quad (26)$$

$$C_{\text{pre}} = H_q [c_K(d_{QK}) S_{T_N}(L) + c_N(d_V) S_{T_K}(L)]. \quad (27)$$

每份 KV 会被多个后续 query 向量读取；“一次 Attention 任务”并不等于每份 KV 只服务一次。

Decode: 从 $L-1$ 追加到可见 L Q_S 与调用数分别取式(15)–(17) 中的 $i = L$ ，写入取式(18)：

$$Q_{S,\text{dec}}^{\text{op}} = H_q (d_{QK} b_q + L b_p), \quad (28)$$

$$Q_{S,\text{dec}}^{\text{loc}} = H_q [d_{QK} b_q c_N(L) + L b_p c_N(d_V)], \quad (29)$$

$$Q_{R,\text{dec}} = H_{kv} (d_{QK} b_K + d_V b_V), \quad (30)$$

$$C_{\text{dec}} = H_q [c_N(L) c_K(d_{QK}) + c_N(d_V) c_K(L)]. \quad (31)$$

写入只计新 token，已有 $L-1$ 个 token 是窗口初态。对每个分项和汇总，RI 都是其累计 Q_S/Q_R ；op、loc 不能混用，分项 RI 不做算术平均。

8 合成例、独立核验与适用条件

不等宽 GQA 例 取 $H_q = 4, H_{kv} = 2, d_{QK} = 3, d_V = 5, L = 7$, 测试 tile 为 3×2 , 所有数值每元素 1 Byte。 $P(7) = 28$ 、 $S_3(7) = 12$ 、 $S_2(7) = 16$ 。

窗口	分项	Q_S^{op} (Byte)	Q_S^{loc} (Byte)	Q_R (Byte)	tile 调用
Prefill	QK	84	144	42	96
	AV	112	224	70	128
	合计	196	368	112	224
Decode	QK	12	36	6	24
	AV	28	56	10	32
	合计	40	92	16	56

Prefill 的局部 RI 为 $368/112 = 23/7$, Decode 为 $92/16 = 23/4$ 。Prefill 分项 RI 的算术平均为 $\frac{1}{2}(144/42 + 224/70) = 116/35$, 并不等于 $23/7$; 应先相加字节再求商。最终只有 $2(7 \times 3 + 5 \times 7) = 112$ 个 resident 元素, 不随 $g = 2$ 复制成 224 个。

窗口衔接 相同组织下, Prefill 到 6 的局部输入/写入/调用为 (276, 96, 168)。追加第 7 个 token 的差量为 (92, 16, 56), 与 Decode 可见 7 的结果逐项一致。一般有

$$\mathcal{D}_{\text{pre}}(L) - \mathcal{D}_{\text{pre}}(L - 1) = \mathcal{D}_{\text{dec}}(L), \quad \mathcal{D} \in \{Q_S^{\text{op}}, Q_S^{\text{loc}}, Q_R, C_{\text{eval}}\}. \quad (32)$$

该差分适用于需求与调用数, **不适用于 RI 相减**。从空 KV 逐 token 执行并累计 Decode 事件, 即得到这里的 Prefill。

独立检查方式 shared/scripts/counting.py 实现闭式公式; check_counting.py 另写分块求和及显式事件枚举, 不复用闭式求值函数作为预期值。显式枚举按输入身份、tile 接收端、求值编号和 resident 元素写入建立事件集合, 逐项验证是否引用已建立的 K/V。检查包含矩形、两方向非整除尾块、静态、一次装载多次使用、每次重载、额外 Q gate、分支共享、GQA、不等宽 QK/V、 $L = 1$ 与跨 128 边界, 以及精确 Prefill/Decode 衔接。Table II(a) 全部 10 格另由独立 tile 求和复算。

集中的适用条件 这是一套单一、可复算的逻辑映射: 足够驻留空间、固定 tile 坐标、独立语义 bank、数字域累加与非矩阵交接。角色字节宽度可以变化, 但必须保持相同计量与对应能力配置; 若需要多个 Task I 服务组合才能实现其它精度, 应重新声明映射。K 行追加与 V 列追加的写入形状已经保留; 页/块擦写、内部读改写及最小事务约束不在这里凭空扩大逻辑 Q_R , 未来介质映射须据真实写服务另做代价/可行性判断。本轮不从总需求推算延迟、token/s、端到端性能或硬件适合度。

9 复现入口、证据与停止点

交付与生成关系

- 本文: `shared/tex/counting_method.zh.tex`; 独立 PDF 位于 `shared/output/counting_method.zh.pdf`。
- 机器约定: `shared/data/conventions.json`, 记录角色精度、双边界、窗口、语义 bank、状态方向及公式输入。
- 合成检查: `shared/data/synthetic_checks.json`, 原始定义与结果可逐项对应本稿例子。
- Table II(a): `table_IIa/data/results.json`、独立表格片段及英文 PDF; 所有数值由 `generate_IIa.py` 生成。
- 原始模型数据与资料仍由 `data/models.json`、`data/sources.json` 和 `literature/` 提供, 不改写 Step 1 原生参数。

所有路径在未特别说明时相对本任务目录。执行

```
TASK2_PYTHON=/opt/anaconda3/bin/python \
sh tasks/task2_table_II_workloads/shared/scripts/build.sh
```

先复算生成数据的一致性, 再分别编译中文方法与英文表格; 检查默认不覆盖数据, 刷新需显式使用 `--emit`。页面渲染与检查记录保存在各自 `tmp/pdfs/` 和 `shared/data/pdf_qa.json`。

审阅重点 本轮建议确认三项共同选择: INT8 的分角色接口参考; 独立语义 bank (不跨分支合并尾块); 从空 KV 逐前缀建立并求值的因果组织。这些是已落实的主参考, 不要求先枚举多种优化调度。Step 3 与六模型 Table II(b) 正式数值仍未启动。

参考文献

- [1] 本仓库, *CIM Roofline* 计量约定, `docs/MODEL_CONVENTIONS.md`; Task I 共享外围参考与十例审阅, `tasks/task1_table_I_NVM/analysis/`。本轮保持不变。
- [2] J. Ainslie et al., *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, arXiv:2305.13245v3, 2023, p.2, §2.2/Fig.2。本地: `literature/00_shared/raw/2305.13245v3.pdf`。
- [3] A. Vaswani et al., *Attention Is All You Need*, arXiv:1706.03762v7, pp.4–5, §3.2。本地: `literature/00_shared/raw/1706.03762v7.pdf`。
- [4] N. Shazeer, *GLU Variants Improve Transformer*, arXiv:2002.05202v1, pp.1–2, §2。本地: `literature/00_shared/raw/2002.05202v1.pdf`。
- [5] Transformers 固定 commit 2c4914f, `modeling_qwen3_5.py` 第 749–821 行, `modeling_qwen3_5_moe.py` 第 751 行起。原件位于 `literature/00_shared/raw/transformers/`; Q 与输出 gate 的投影/切分见 Attention 构造与 forward。
- [6] Step 1 固定六模型结构卡与 `data/models.json`。Ling/MiMo 原生实现位于各自资料包, MiMo 的 $d_{QK} \neq d_V$ 、Value 缩放及 K/V cache 更新均已定位。上游格式用于结构依据, 本稿的 INT8 与端口映射是本文另选参考。